

MAKERERE UNIVERSITY
COLLEGE OF COMPUTING AND INFORMATION SCIENCES
SCHOOL OF COMPUTING AND INFORMATICS TECHNOLOGY

BSE4104: Emerging Trends in Software Engineering

8-Week AI-Native & Agentic Engineering Capstone

UniSupport AI

A Bounded Agentic University Student-Support Case System

WEEK 1 DELIVERABLES — Problem Framing and AI-Native Requirements

Group Name / Number: Group L

Team Members:

Name	Registration No
Musheija Abraham	23/U/12139/EVE
Okema Paul Mark	23/U/16648/EVE
Akakikunda Benitah	23/U/05477/EVE
Kizito Ryan	23/U/10160/EVE
Mwesigwa Sean	21/U/12248/PS

0. Use Case Selection Note

Two candidate proposals were considered before Week 1 submission: (a) a broad “AI University Student Assistant” spanning academic, services, deadline and escalation domains with 3-4 specialised agents, and (b) a bounded “University Student-Support Case Agent” limited to grounded Q&A, timetable/case lookup and ticket creation/routing.

This charter proceeds with option (b) because it is one of the use cases explicitly pre-approved in the capstone brief (Section 3, “University student-support case agent”), whereas option (a)’s multi-agent, multi-domain scope risks breaching the brief’s explicit rule against proposals that behave like a “generic assistant for everything.” Option (b) is also small enough to fully implement, test and explain within eight weeks, while still exercising every required capability (RAG, tools, a bounded agent loop, memory, evaluation, observability and guardrails).

1. Minimum Proposal Statement

Our system helps university students complete the task of getting accurate, grounded answers to academic/support questions and, where self-service is insufficient, creating and tracking a support case. AI is used for retrieval-augmented reasoning over approved documents, intent/category classification, and routing recommendations. Deterministic software remains responsible for ticket-ID generation, case/timetable data storage, student authentication, and the department-routing rule table. The agent may use approved tools (`search_university_documents`, `check_timetable`, `check_case_status`, `create_support_ticket`, `route_support_case`) but may not decide admissions, grades, disciplinary outcomes or fee waivers. We will build and evaluate the system using synthetic student/case/timetable records and public Makerere course-handbook-style documents.

2. Project Charter

2.1 Problem

University students routinely need answers to procedural questions (registration steps, retake rules, examination policy, where to submit a complaint) and need to report problems (e.g. “my course registration failed”) that require follow-up. Official answers exist in scattered handbooks and are otherwise only available by physically visiting an office or emailing a department, and once a student reports a problem there is no simple way for them to check whether it has been picked up.

2.2 Target User

Primary: undergraduate students seeking procedural information or needing to log and track a support issue.
Secondary: support-desk / departmental staff who receive and resolve routed cases.

2.3 Current Pain Point

- Procedural answers require manually searching PDFs or visiting an office, at a real time cost to students.
- Support requests sent informally (email, in person) are easy to lose track of, with no case ID or status to check.
- Students re-explain their issue every time they follow up, because nothing remembers the case they already opened.

2.4 Why AI (AI Value)

A foundation model can (a) answer in natural language while staying grounded in retrieved, approved documents rather than guessing, (b) classify an incoming free-text complaint into a support category with enough confidence to route it, and (c) hold short-term conversational memory so a follow-up like “has it been resolved?” resolves to the right case. None of this requires the model to make the underlying decision — it drafts and retrieves; deterministic logic and, where required, a human, decide.

2.5 Scope (Single Primary End-to-End Workflow)

A student asks a question. The agent retrieves grounded context from an approved document corpus and answers directly when it can. If the request is a timetable or existing-case query, the agent calls the matching tool. If the issue cannot be resolved from documents or existing tools, the agent creates a support ticket, routes it to the correct department using a deterministic rule table, and remembers the resulting case ID as the active case for the remainder of the session. High-impact categories (admissions, grading, discipline, fees) are never resolved by the agent — they are always handed to a human.

2.6 Assumptions

- Only public, team-created or synthetic documents and records are used — no real student data.
- A small, fixed synthetic corpus (course handbook excerpts, support-procedure pages) and synthetic case/timetable/student tables are sufficient to demonstrate the workflow.
- One LLM provider accessible to the team (subject to Week 2 Model Selection Note) is available for the whole project.

2.7 Constraints

- Eight-week timeline; the team must finish, test and explain the system, not maximise features.
- No confidential or real institutional data; no production integration with Makerere's actual systems.
- The agent must never autonomously resolve admissions, grading, disciplinary or fee matters — these always require human approval.
- At least two tools by Week 4, one bounded agent loop by Week 5, one justified memory mechanism by Week 6, and a 30-scenario evaluation by Week 7.

2.8 Success Criteria

- A grounded answer is produced with a cited source for at least 90% of in-corpus questions in the Week 3 RAG evaluation.
- A new support issue is correctly turned into a ticket and routed to the right department in the Week 4-5 tool/agent tests.
- A follow-up question correctly resolves to the previously opened case in the Week 6 memory demonstration.
- Zero autonomous resolutions of admissions/grading/disciplinary/fee requests across the Week 7 guardrail test set.

3. User Stories and Acceptance Criteria

#	User Story	Acceptance Criteria
US1	As a student, I want to ask questions about university procedures so that I get an accurate, grounded answer.	Answer is generated only from the approved document corpus and cites the source document/section; if nothing relevant is retrieved, the agent states the information was not found rather than guessing.
US2	As a student, I want to check my course timetable so that I know when and where my classes are.	check_timetable(course_code) returns the correct synthetic schedule for a valid code; an invalid/unknown code returns a clear error message, not a fabricated timetable.
US3	As a student, I want to check the status of an existing support case so that I know if it has been handled.	check_case_status(case_id) returns the correct stored status; an unknown case ID returns a clear “not found” message.

#	User Story	Acceptance Criteria
US4	As a student, I want to open a support ticket when my question cannot be answered from documents or existing tools.	create_support_ticket() generates a unique case ID, stores category and description, and confirms the case ID back to the student.
US5	As a student, I want my new ticket automatically sent to the right department so that I do not have to know the university's internal structure.	route_support_case() assigns a department using the deterministic rule table; a category with no confident match is flagged for human review rather than guessed.
US6	As a student, I want the agent to remember my active case within our conversation so I do not have to repeat my case ID.	A follow-up message referring to “it”/“my case” resolves to the case ID stored in session memory for that conversation.
US7	As a student, I want to be told clearly when the agent cannot help so that I am not misled by a guess.	For out-of-scope or high-impact requests, the response explicitly states the limitation and names the correct human office, rather than attempting an answer.
US8	As a university stakeholder, I want the agent blocked from making admissions, grading, disciplinary or fee decisions.	Every test prompt in the AI Boundary Matrix's 'requires human approval' category is refused/escalated by the agent, verified by a dedicated boundary test set.
US9	As a student, I want an ambiguous request clarified before the agent acts so it does not do the wrong thing.	When intent confidence is low (e.g., missing course code or case ID), the agent asks one clarifying question instead of calling a tool or guessing.
US10	As a developer, I want tool failures handled safely so that the system never presents a hallucinated result.	A simulated tool/database failure returns a defined fallback message and is logged, without the LLM inventing a plausible-looking answer.
US11	As support staff, I want a record of what the agent decided so that I can audit and verify its actions.	Each session logs the detected intent, the tool called (if any), the sources retrieved, and the final response returned to the student.

4. AI Boundary Matrix

This matrix defines what the LLM/agent is permitted to decide, what must remain deterministic software logic, and what must always be routed to a human before any consequential action is taken.

AI May Do	Stays Deterministic	Requires Human Approval
Answer questions using retrieval-augmented generation grounded in the approved document corpus.	Generating and storing ticket IDs; all case/timetable database reads and writes.	Any admissions, grading, disciplinary or fee-related determination.
Classify/triage the likely category of a new support issue from the student's free-text description.	Authenticating the student's identity before returning case data.	Closing or escalating a case that has been classified as sensitive/high-impact.
Decide which tool to call (search documents, check timetable, check case status, create ticket) based on the request.	The department-routing rule table (which category maps to which department).	Overriding the deterministic routing suggestion produced by the rule table.
Draft a concise ticket description/summary from the student's own words.	Logging/audit-trail creation for every request.	Deleting or modifying an existing case record.
Ask a clarifying question when intent or required parameters are ambiguous.	Enforcing the tool allow-list and any rate/iteration limits on the agent loop.	Releasing any information that would require confirming a student's real academic record beyond the synthetic dataset.

5. Initial Architecture / Context Diagram

The diagram below shows the Week 1 context: a student interacts with a single LLM-based support agent that can call a RAG tool, two data-lookup tools, and session memory, all gated by a guardrail check before any ticket is created and routed. High-impact requests are blocked from autonomous resolution and handed to human support staff.

UniSupport AI — Bounded Agent Workflow (Week 1 Context Diagram)



